

M.Sc. Programme in Scientific Computing
Department of Scientific Computing, Modeling & Simulation
Savitribai Phule Pune University, Pune – 411 007

Course: SC-551 PPL-II C++

Max Marks: 175

Curriculum: 23NEP | **Credits:** 4CR

Duration: 3 Hours

Academic Year: 2025-26

Date: _____

General Instructions:

- Solve any ONE problem from each question unless specified otherwise.
 - A Makefile is mandatory for compiling C++ code. Absence of Makefile = failure in exam.
 - Code must be modularized. Entire code should NOT be written in a single function.
 - There should be one file named **main.cpp** which contains only the main function (not more than 10 lines of code) which instantiates your class. Each class should be written in two files **.h** and **.cpp**.
 - To obtain good grades, use C++ features such as exception handling, STL, etc.
 - Your program must work for any valid input, not just the test cases provided.
 - Do NOT make any sub-folders in the submit folder. Submit only plain text files.
 - Questions labelled (*Haskell Converted*) are originally Haskell problems re-stated as C++ problems.
 - Wrong/irrelevant answers are not necessarily entitled to marks.
-

Q1 (Maximum Marks: 25)

Design, develop and test a C++ program to accept a positive integer and a base of number system, both in decimal notation. Your program should convert the first number to the notation using the second number as the base. Use OOAD principles — define an appropriate class encapsulating the conversion logic.

Test Cases:

Input: 173 2 Output: 10101101

Input: 47 16 Output: 2F

Input: 2578346 10 Output: 2578346

Q2 (Maximum Marks: 25)

Design, develop and test a C++ program to accept a positive real number in the form **F.I** (float form). Treating **F** as the fractional part and **I** as the integral part, print the number in the form **GH.J** where **H** is a sequence of hashes ('#') such that the decimal point always occurs at the 6th position from the left, **G** is I with only the significant digits, and **J** is F with only the significant digits. Terminate the output with a newline. Assume the fractional part F always has no leading zeroes and both F and I have at most 5 digits.

Test Cases:

Input: 10000.6660 Output: 6660#.1

Input: 990.0080 Output: 80####.99

Input: 101.11111 Output: 11111.101

Input: 31657.0 Output: #####.31657

Q3 (Maximum Marks: 25)

Design, develop and test a C++ program for reading a text file and displaying statistics such as the number of lines, number of words, number of characters and number of sentences contained in that file.

The number of characters should include characters like white spaces and text control characters.

Test Cases:

Input: File contains —

*I am a C programmer. My programs work great and they hardly
have any logical errors. Pointer is my favourite data type.*

Output: number of lines : 2 | number of words : 22 | number of characters : 121 | number of sentences : 3

Q4 (Maximum Marks: 25)

Given the coefficients **a, b, c, h, g, f** in the equation $ax^2 + by^2 + 2hxy + 2gx + 2fy + c = 0$, calculate the discriminant $\Delta = \det([a \ h \ g; h \ b \ f; g \ f \ c])$ and decide what the equation represents. When $\Delta \neq 0$: $ab - h^2 > 0 \rightarrow$ ellipse/circle, $= 0 \rightarrow$ parabola, $< 0 \rightarrow$ hyperbola. When $\Delta = 0$: $ab - h^2 > 0 \rightarrow$ two imaginary lines, $= 0 \rightarrow$ two real parallel lines, $< 0 \rightarrow$ two real intersecting lines.

Test Cases:

Input: 1 1 0 0 0 0 Output: two imaginary lines

Input: 16 9 100 12 -40 -70 Output: parabola

Input: 9 4 0 15 -8 -40 Output: circle

Q5 (Maximum Marks: 25)

Design, develop and test a C++ program to accept a positive real number in the form **I.F** (float form). Convert the fractional part F to its reduced ratio form **N/D**, where N and D are co-prime integers (no common factor other than 1). If the integral part is non-zero, print it as well.

Test Cases:

Input: 9.25 Output: 9 + 1 / 4

Input: 16.375 Output: 16 + 3 / 8

Input: 0.7777 Output: 7777 / 10000

Q6 (Maximum Marks: 25)

Design a C++ class to factorize a quadratic polynomial of the form $ax^2 + bx + c = (x + z_1)(x + z_2)$, given a, b, c as input. Note that z_1, z_2 are, in general, complex numbers. The class should handle real as well as complex roots.

Test Cases:

Input: 5 4 -1 Output: (5x-1)(x+1)

Input: 1 0 25 Output: (x+5i)(x-5i)

Input: 1 4.5 5 Output: (x+2.5)(x+2)

Q7 (Maximum Marks: 25)

Design, develop and test a C++ class to find all symmetric dates in the current century (2001–2100). A date is symmetric when its digit sequence reads the same forwards and backwards (e.g., 05-02-2050 is symmetric because the digit sequence 0 5 0 2 2 0 5 0 is a palindrome). Print all such dates in the format DD-MM-YYYY.

Test Cases:

Sample symmetric date: 05-02-2050

Program should enumerate and print ALL symmetric dates in 2001–2100.

Q8 (Maximum Marks: 25)

A magic square of order N is an $N \times N$ grid filled with distinct integers 1 to N^2 such that every row, column and both diagonals sum to $N(N^2+1)/2$. Design, develop and test a C++ class **CMagicSquareGenerator** that takes in an odd N and has two methods **generate** and **print**. Use the following algorithm:

1. Write 1 in the center cell of the first (top) row. Insert remaining numbers 2, 3, ..., N^2 in increasing order into the remaining cells by moving one step at a time in the north-east direction.
2. Movement rules: (a) If the destination cell is empty $\rightarrow$ insert the number there. (b) If movement takes you out at the top of a column $\rightarrow$ insert in the bottom cell of that column. (c) If movement takes you out at the right of a row $\rightarrow$ insert in the first cell (left) of that row. (d) If the destination cell is already filled $\rightarrow$ insert immediately below the current cell. (e) If movement takes you to the upper-right corner $\rightarrow$ insert immediately below the upper-right corner cell.

Test Cases:

Output for N=3: Magic square with magic constant 15.

Your program should generate and print correct magic squares for at least N = 3, 5 and 7.

Note: This algorithm works only for odd N.

Q9 (Maximum Marks: 25)

Design, develop and test a C++ class to find all three angles of a triangle given the lengths of its three sides a, b, c. If a triangle cannot be formed with the given side lengths, print an appropriate error message. You may use the Law of Cosines: $c^2 = a^2 + b^2 - 2ab \cos C$ (and similar expressions for the other angles).

Test Cases:

Input: 16.5 17 21 Output: 50 52 78 degrees

Input: 5 5 10 Output: Invalid input

Input: 3 4 5 Output: 37 53 90 degrees

Q10 (Maximum Marks: 25)

Develop a C++ program to find the function value at any point $x \in [x_0, x_n]$ given a set of 2D points $\{(x_i, f(x_i))\}$, $i = 0, 1, 2, \dots, n$. Use Lagrangian interpolation:

$$f(x) = \sum_{i=0}^n f(x_i) \prod_{j=0, j \neq i}^n (x - x_j) / (x_i - x_j)$$

The class should accept n+1 data points and evaluate f at any queried x value.

Q11 (Maximum Marks: 25)

Design, develop and test a C++ class to determine if a given 4-digit positive integer represents a leap year or not. The class must validate the input (must be a 4-digit integer) and throw an appropriate exception for invalid input.

Test Cases:

Input: 2000 Output: Leap year

Input: 1900 Output: Not a leap year

Input: 2024 Output: Leap year

Input: 123 Output: Invalid input (not a 4-digit number)

Q12 (Maximum Marks: 25) [Haskell Converted]

[Haskell Converted] Design, develop and test a C++ class to find the GCD (Greatest Common Divisor) of two positive integers given by the user. Implement the Euclidean algorithm as a method of the class. The class should validate inputs and handle edge cases (e.g., $\text{GCD}(a, 0) = a$).

Recurrence: $\text{GCD}(a, 0) = a$ $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$

Test Cases:

Input: 48 18 Output: GCD = 6

Input: 100 75 Output: GCD = 25

Input: 7 0 Output: GCD = 7

Q13 (Maximum Marks: 25) [Haskell Converted]

[Haskell Converted] Design, develop and test a C++ class to find the LCM (Least Common Multiple) of two positive integers given by the user. Use the relation $\text{LCM}(a, b) = (a \times b) / \text{GCD}(a, b)$. The class should reuse the GCD logic (either by inheritance or composition).

Test Cases:

Input: 4 6 Output: LCM = 12

Input: 15 20 Output: LCM = 60

Input: 7 9 Output: LCM = 63

Q14 (Maximum Marks: 25) [Haskell Converted]

[Haskell Converted] Design, develop and test a C++ class to find both roots of a quadratic equation $ax^2 + bx + c = 0$. The class should take values of a, b, c from the user and correctly handle real distinct roots, real equal roots, and complex conjugate roots. Use appropriate exception handling for the degenerate case $a = 0$.

Test Cases:

Input: a=1 b=-5 c=6 Output: Roots are 3.00 and 2.00

Input: a=1 b=0 c=1 Output: Roots are $0 + 1.00i$ and $0 - 1.00i$

Input: a=1 b=-2 c=1 Output: Roots are equal: 1.00

Q15 (Maximum Marks: 25) [Haskell Converted]

[Haskell Converted] Design, develop and test a C++ class to find all three angles of a triangle given its three sides a, b, c entered by the user. The class should use the Law of Cosines and the Law of Sines to compute all three angles. Angles should be reported in degrees rounded to the nearest integer. If no valid triangle can be formed, display an error.

Test Cases:

Input: 3 4 5 Output: 37 53 90 degrees

Input: 16.5 17 21 Output: 50 52 78 degrees

Input: 1 1 100 Output: Invalid input

Q.No	Topic	Marks	Type
Q1	Number Base Conversion	25	C++
Q2	Float Formatting (F.I → GH.J)	25	C++
Q3	File Statistics	25	C++
Q4	Conic Section Classifier	25	C++
Q5	Fraction Reduction (I.F → N/D)	25	C++
Q6	Quadratic Factorization	25	C++
Q7	Symmetric Dates Finder	25	C++

Q8	Magic Square Generator	25	C++
Q9	Triangle Angles from Sides	25	C++
Q10	Lagrangian Interpolation	25	C++
Q11	Leap Year Checker	25	C++
Q12	GCD of Two Numbers	25	Haskell Converted
Q13	LCM of Two Numbers	25	Haskell Converted
Q14	Quadratic Roots (both roots)	25	Haskell Converted
Q15	Triangle Angles (Haskell Converted)	25	Haskell Converted
Total		375	

*Note: Questions Q12–Q15 are originally Haskell problems, re-stated here as C++ problems. Solve any problem you are assigned.
Total marks shown are for the full paper; individual exam marks are as per the exam instructions.*